

SYSCALL SENTINEL

Functional Requirements Document

Alumno: Jesús Miguel Reza Zatarain | **ID:** 0276971

Materia: Sistemas Operativos | **Fecha:** Mayo 2026

PROYECTO FINAL

RESUMEN EJECUTIVO

Syscall Sentinel es una herramienta de ciberseguridad defensiva desarrollada en C++ para Linux que monitorea en tiempo real las llamadas al sistema (syscalls) de cualquier programa durante su ejecución. El proyecto resuelve la necesidad de detectar comportamientos maliciosos a nivel del sistema operativo sin necesidad de analizar el código fuente del programa vigilado, observando únicamente sus interacciones con el kernel.

El proyecto va dirigido a estudiantes de ingeniería en ciberseguridad y sistemas operativos que buscan comprender cómo los programas interactúan con el kernel de Linux, así como a profesionales interesados en herramientas de análisis de comportamiento en entornos controlados. La razón de ser del proyecto es aprender de forma práctica los conceptos de procesos, comunicación entre procesos y syscalls mediante la implementación de una herramienta real de seguridad defensiva, aplicando directamente conceptos del curso como `fork()`, `execvp()`, `waitpid()` y `ptrace()`.

Se utilizó `ptrace()` como biblioteca principal porque es la única interfaz nativa del kernel de Linux que permite a un proceso observar y controlar la ejecución de otro proceso sin modificar su código, siendo la misma base tecnológica que utilizan herramientas profesionales como `strace`, `gdb` y `Falco`.

TIEMPO DE DESARROLLO

~16

HORAS TOTALES DE DESARROLLO

2h

DISEÑO

8h

IMPLEMENTACIÓN

4h

PRUEBAS

2h

DOCUMENTACIÓN

TECNOLOGÍAS UTILIZADAS

LENGUAJE DE PROGRAMACIÓN

- C++17 — lenguaje principal
- Bash — scripts de prueba

COMPILADOR Y BUILD

- g++ 11.4+ (GNU Compiler)
- GNU Make — automatización

APIS UNIX / LINUX

- ptrace() — monitoreo de procesos
- fork() — creación de procesos
- execvp() — ejecución de programas
- waitpid() — sincronización
- PTRACE_GETREGSET — lectura de registros
- PTRACE_PEEKDATA — lectura de memoria

HERRAMIENTAS EXTRA

- Docker — contenedor Linux
- Ubuntu 22.04 — sistema operativo
- VS Code — editor de código
- GitHub — control de versiones

LIBRERÍAS ESTÁNDAR C++

- <iostream> — entrada/salida
- <fstream> — manejo de archivos
- <string> — cadenas de texto
- <vector> — colecciones
- <sstream> — streams de texto
- <ctime> — timestamps

HEADERS DEL SISTEMA

- <sys/ptrace.h>
- <sys/wait.h>
- <sys/syscall.h>
- <sys/uio.h>
- <signal.h>
- <unistd.h>

DESARROLLO DEL PROYECTO — PASOS Y TAREAS

1 **Diseño de la arquitectura padre-hijo**

Se definió el modelo de dos procesos: el proceso padre actúa como monitor y el proceso hijo ejecuta el programa objetivo. Esta separación es necesaria porque ptrace solo puede observar procesos externos, no funciones internas.

2 **Implementación del módulo Logger**

Se creó la clase Logger para centralizar la salida de alertas e información. Escribe simultáneamente en consola y en el archivo `sentinel.log` con timestamp en cada entrada.

3 **Implementación del módulo Rules**

Se definieron las reglas de detección en una clase separada (separación de responsabilidades). Contiene listas de archivos críticos, archivos sensibles y binarios peligrosos. Evalúa cada syscall y retorna un nivel de alerta: NONE, MEDIA, ALTA o CRÍTICA.

4 **Implementación del módulo Monitor con ptrace**

El corazón del sistema. Usa `PTRACE_SYSCALL` para detener al proceso hijo en cada entrada y salida de syscall. Lee los registros del CPU con `PTRACE_GETREGSET` (ARM64) para identificar el número de syscall y sus argumentos. Usa `PTRACE_PEEKDATA` para leer strings desde la memoria del proceso hijo.

5 **Implementación del punto de entrada (main)**

Orquesta el ciclo de vida completo: valida argumentos, crea el Logger, ejecuta `fork()`, en el hijo llama `PTRACE_TRACEME` seguido de `execvp()`, y en el padre instancia el Monitor.

6 **Creación de programas de prueba**

Se desarrollaron dos programas de prueba: `safe_program` que solo crea y lee un archivo en `/tmp` (sin alertas), y `suspicious_program` que accede a `/etc/passwd`, cambia permisos, borra un archivo y ejecuta `/bin/sh` (genera 4 alertas).

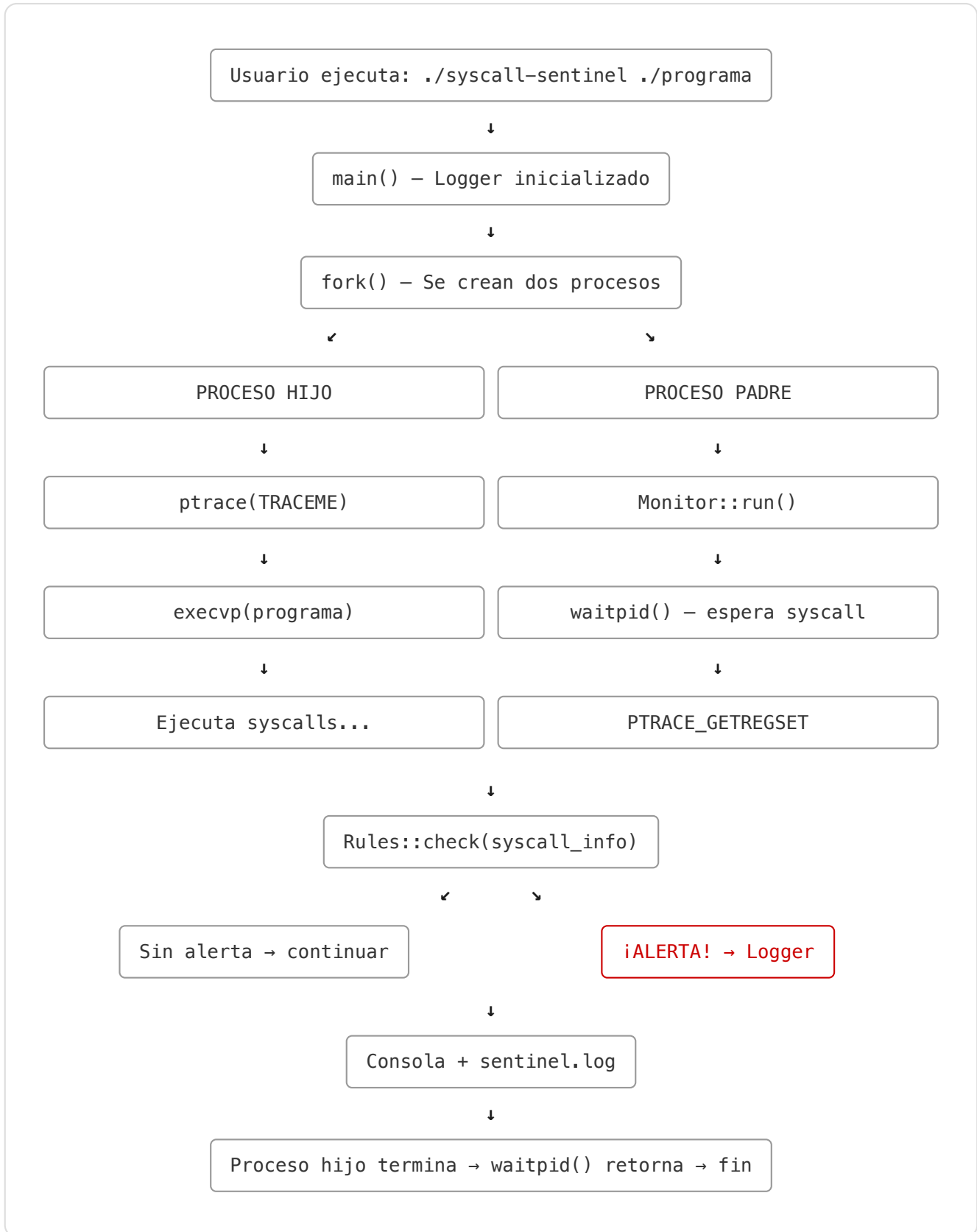
7 **Contenedorización con Docker**

Se creó un Dockerfile basado en Ubuntu 22.04 con g++ y make. El `docker-compose.yml` configura los permisos necesarios: `SYS_PTRACE` capability y `seccomp=unconfined` para que ptrace funcione dentro del contenedor.

8 **Pruebas y validación**

Se verificó que el programa seguro no genera alertas y que el programa sospechoso genera exactamente 4 alertas (ALTA por `/etc/passwd`, ALTA por `chmod`, MEDIA por `unlink`, ALTA por `execve` a `/bin/sh`). Se verificó la persistencia del log.

DIAGRAMA DE FLUJO DEL SISTEMA



NIVELES DE ALERTA

⚠ **MEDIA**

Comportamiento inusual pero no necesariamente malicioso.
Ejemplo: borrado de archivos con `unlink()`.

⚠⚠ **ALTA**

Comportamiento muy sospechoso.
Ejemplo: acceso a `/etc/passwd`, ejecución de shell, cambio de permisos.

🔴 **CRÍTICA**

Acción extremadamente peligrosa.
Ejemplo: acceso a `/etc/shadow`, `/etc/sudoers`, claves SSH.

ESTRUCTURA DEL PROYECTO

ARCHIVO

RESPONSABILIDAD

`main.cpp`

Punto de entrada. Gestiona `fork()`, `execvp()` y ciclo de vida.

`monitor.h` / `monitor.cpp`

Bucle `ptrace`. Intercepta syscalls y lee registros/memoria.

`rules.h` / `rules.cpp`

Lógica de detección. Define qué es sospechoso y con qué nivel.

`logger.h` / `logger.cpp`

Salida de alertas a consola y archivo `sentinel.log`.

`Makefile`

Automatización de compilación y ejecución.

`Dockerfile`

Contenedor Linux con `g++` y `make` para portabilidad.

`docker-compose.yml`

Configuración de permisos `SYS_PTRACE` para Docker.

`test_programs/safe_program.cpp`

Programa de prueba sin comportamiento sospechoso.

`test_programs/suspicious_program.cpp`

Programa de prueba con acciones detectables.

REQUERIMIENTOS FUNCIONALES

#

REQUERIMIENTO

RF-01 Ejecutar cualquier programa pasado como argumento en modo monitoreado.

RF-02 Interceptar syscalls: `openat`, `execve`, `unlinkat`, `fchmodat`, `clone`.

RF-03 Detectar acceso a archivos críticos (`/etc/shadow`, `/etc/sudoers`).

RF-04	Detectar acceso a archivos sensibles (/etc/passwd, /etc/group).
RF-05	Detectar ejecución de shells e intérpretes (/bin/sh, python, perl).
RF-06	Detectar borrado de archivos con unlink.
RF-07	Detectar cambios de permisos con chmod.
RF-08	Detectar creación excesiva de procesos (fork bomb).
RF-09	Clasificar alertas en niveles MEDIA, ALTA y CRÍTICA.
RF-10	Persistir todas las alertas en sentinel.log con timestamp.

Jesús Miguel Reza Zatarain — ID: 0276971 — Sistemas Operativos — Mayo 2026

Syscall Sentinel | syscall-sentinel | C++ + ptrace + Docker